

RTL Design Sherpa

Lattice Generator Architecture Specification 1.2

October 13, 2026

Table of Contents

1 Lattice Gas Index....13

2 Lattice Hardware Architecture Specification Index....14

2.1 Document Organization....15

2.1.1 Front Matter....15

2.1.2 Chapter 1: Introduction....15

2.1.3 Chapter 2: System Overview....15

2.1.4 Chapter 3: Architecture....15

2.1.5 Chapter 4: Interfaces....15

2.1.6 Chapter 5: Performance....15

2.1.7 Chapter 6: Integration....15

2.2 Related Documentation....17

3 Lattice Micro-Architecture Specification Index....18

3.1 Document Organization....19

3.1.1 Main Documentation....19

3.2 Chapter 1: Architecture....20

3.3 Chapter 2: Address Decode and Arbitration....21

3.4 Chapter 3: RTL Generator....22

3.5 Quick Navigation....23

3.5.1 For New Users....23

3.5.2 For Integration....23

3.5.3 Common Questions....23

3.6 Visual Assets....24

3.6.1 Architecture Diagrams....24
3.6.2 Timing Diagrams....24
3.7 Component Overview....25
3.7.1 Key Features....25
3.7.2 Pre-Generated Variants....25
3.7.3 Design Philosophy....25
3.8 Related Documentation....26
3.8.1 Companion Specifications....26
3.8.2 Project-Level....26
3.8.3 Test Infrastructure....26
3.8.4 RTL....26
3.9 Version History....27
4 Product Requirements Document (PRD)....28
4.1 Lattice Generator....28
4.2 1. Executive Summary....29
4.2.1 1.1 Quick Stats....29
4.2.2 1.2 Project Goals....29
4.3 2. Key Design Principles....30
4.3.1 2.1 Architecture Philosophy....30
4.3.2 2.2 Design Trade-offs....30
4.4 3. Architecture Overview....31
4.4.1 3.1 Top-Level Block Diagram....31
4.4.2 3.2 Address Mapping....32
4.4.3 3.3 Arbitration Strategy....33

4.5 4. Available Variants....35
4.5.1 4.1 Pre-Generated Modules....35
4.5.2 4.2 Wrapper Modules....35
4.6 5. Functional Requirements....37
4.6.1 FR-1: Arbitrary MxN Configuration....37
4.6.2 FR-2: Round-Robin Arbitration....37
4.6.3 FR-3: Address-Based Routing....37
4.6.4 FR-4: Back-to-Back Transactions....37
4.6.5 FR-5: Configurable Address Map....37
4.7 6. Interface Specifications....38
4.7.1 6.1 Master-Side Interface (APB Slave)....38
4.7.2 6.2 Slave-Side Interface (APB Master)....38
4.8 7. Parameter Configuration....39
4.9 8. Generator Usage....40
4.9.1 8.1 Pre-Generated Variants....40
4.9.2 8.2 Custom Generation....40
4.9.3 8.3 Generator Options....40
4.10 9. Performance Characteristics....41
4.10.1 9.1 Latency....41
4.10.2 9.2 Throughput....41
4.10.3 9.3 Resource Utilization (Estimated)....41
4.11 10. Verification Status....42
4.11.1 10.1 Test Coverage....42
4.11.2 10.2 Test Methodology....42

4.12 11. Integration Guide....43
4.12.1 11.1 Simple Integration (1to4)....43
4.12.2 11.2 Multi-Master Integration (2to4)....43
4.13 12. Design Constraints....45
4.13.1 12.1 Limitations....45
4.13.2 12.2 Assumptions....45
4.14 13. Future Enhancements....46
4.14.1 13.1 Potential Features....46
4.14.2 13.2 Generator Improvements....46
4.15 14. References....47
4.15.1 14.1 Internal Documentation....47
4.15.2 14.2 External Standards....47
4.15.3 14.3 Related Components....47
5 Claude Code Guide: Lattice Component....48
5.1 Quick Context....49
5.2 Critical Rules for This Component....50
5.2.1 Rule #0: This is a GENERATOR, Not Just Modules....50
5.2.2 Rule #1: Address Map is Fixed Per-Slave....50
5.2.3 Rule #2: Know the Pre-Generated Variants....50
5.2.4 Rule #3: Generator Syntax....51
5.3 Architecture Quick Reference....52
5.3.1 Module Organization....52
5.3.2 Block Diagram (2×4 Example)....52
5.4 Common User Questions and Responses....54

5.4.1 Q: “How do I connect a CPU to 4 peripherals?”....54
5.4.2 Q: “I need CPU and DMA to access peripherals. Which crossbar?”....54
5.4.3 Q: “What if I need 3 masters and 8 slaves?”....55
5.4.4 Q: “How does arbitration work?”....55
5.4.5 Q: “Can I change the address map?”....56
5.4.6 Q: “How do I run tests?”....56
5.5 Integration Patterns....58
5.5.1 Pattern 1: Simple Peripheral Interconnect....58
5.5.2 Pattern 2: Multi-Master System....58
5.5.3 Pattern 3: Hierarchical Interconnect....59
5.6 Anti-Patterns to Catch....60
5.6.1 Anti-Pattern 1: Generating When Pre-Generated Exists....60
5.6.2 Anti-Pattern 2: Assuming Custom Per-Slave Sizes....60
5.6.3 Anti-Pattern 3: Not Mentioning Address Map....60
5.6.4 Anti-Pattern 4: Forgetting About Wrappers....60
5.7 Debugging Workflow....62
5.7.1 Issue: Address Not Routing Correctly....62
5.7.2 Issue: Arbitration Not Fair....62
5.7.3 Issue: Back-to-Back Transactions Stalling....62
5.8 Quick Commands....63
5.9 Remember....64
6 Document Information....65
6.1 Lattice Generator Architecture Specification....65
6.2 Revision History....66

6.3 Document Purpose....67
6.4 Intended Audience....68
6.5 Related Documents....69
7 What the Generator Produces....70
7.1 Output artifacts....71
7.2 Architecture in the generated module....72
8 When to Regenerate....73
8.1 Pre-generated vs custom....74
9 Generator Entry Points....75
9.1 Low-level generator....75
9.2 Convenience batch script....76
9.3 Programmatic API....77
10 CLI and Programmatic API....78
10.1 apbx_xbar_generator.py....79
10.2 generate_xbars.py....80
10.3 Programmatic API....81
11 Parameter Semantics....82
11.1 Generation-time parameters....83
11.2 Elaboration-time parameters....84
12 Mixed APB4/APB5 and Parity....85
12.1 Version list rules....86
12.2 PWAKEUP direction and forwarding....87
12.3 Mixed-port sideband behavior....89
12.4 Parity (enable_parity=True)....90

13 Validation Rules..... 92
13.1 Structural checks..... 93
13.2 Address-map checks..... 94
13.3 Parity checks..... 95
14 Pipeline Overview..... 96
14.0.1 Figure 3.1: Lattice Generator Pipeline..... 96
14.1 Stages..... 97
15 Request Decode..... 98
15.1 Offset-based slave selection..... 99
15.2 Decode-miss handling..... 100
15.3 Single-master simplification..... 101
16 Arbitration and Routing..... 102
16.1 Per-slave request vector..... 103
16.2 Round-robin arbitration..... 104
16.3 Command routing..... 105
16.4 Response routing..... 106
16.5 Ready routing..... 107
17 Boundary Module Instantiation..... 108
17.1 Master-side modules..... 109
17.2 Slave-side modules..... 110
17.3 Parity tie-offs..... 111
17.4 PWAKEUP wiring..... 112
17.5 1×1 passthrough..... 113
18 Directory Layout..... 114

19 Module Header and Parameters....115
19.1 Address-map comment....116
19.2 Port naming convention....117
20 cmd/rsp Fabric....118
20.1 Fabric width invariance....120
21 Wrapper Scaffolds....121
21.1 Filelist convention....122
22 Full Regeneration Rule....123
22.1 Regeneration command....124
22.2 Do not hand-edit generated files....125
23 Pre-Generated Fixtures....126
23.1 When to add a new fixture....127
24 DV Suite....128
24.1 Test list....129
24.2 Running the suite....130
24.3 Adding a new test....131
25 Lint Checks....132
25.1 Width checks....133
25.2 Unused signal checks....134
25.3 Synthesis smoke test....135
26 Adding a New Standard Variant....136
26.1 Naming convention....137
27 Adding a New Generator Option....138
27.1 Backward compatibility....139

28 Adding APB5 Parity or Sideband....140
28.1 Widening sideband signals....141
28.2 Changing parity behavior....142
29 Testing an Extension....143
29.1 Minimal test plan....144
29.2 Example: adding a custom variant test....145
29.3 Commit discipline....146

List of Figures

Figure 3.1: Lattice Generator Pipeline..... 95

List of Tables

No tables in this document.

1 Lattice Gas Index

Generated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Lattice Hardware Architecture Specification Index

Version: 1.2 **Date:** 2026-10-12 **Purpose:** High-level hardware architecture specification for Lattice component

---

2.1 Document Organization

Note: All chapters are linked below; this index doubles as the manifest for automated document generation.

2.1.1 Front Matter

- Document Information

2.1.2 Chapter 1: Introduction

- Purpose and Scope
- Document Conventions
- Definitions and Acronyms

2.1.3 Chapter 2: System Overview

- Use Cases
- Key Features
- System Context

2.1.4 Chapter 3: Architecture

- Block Diagram
- Data Flow
- Address Mapping

2.1.5 Chapter 4: Interfaces

- Master-Side APB Interfaces
- Slave-Side APB Interfaces
- Clock and Reset

2.1.6 Chapter 5: Performance

- Throughput Characteristics
- Latency Analysis
- Resource Estimates

2.1.7 Chapter 6: Integration

- System Requirements
- Parameter Configuration
- Verification Strategy

2.2 Related Documentation

- **Lattice MAS** - Micro-Architecture Specification (detailed block-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide

Last Updated: 2026-10-12 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 Lattice Micro-Architecture Specification Index

Component: Lattice (MxN Interconnect) **Version:** 1.2 **Date:** 2026-10-12 **Purpose:** Detailed micro-architecture specification for Lattice component

---

3.1 Document Organization

This specification covers the Lattice: a parametric MxN interconnect that routes multiple APB masters to multiple APB slaves with automatic address-based routing and round-robin arbitration.

3.1.1 Main Documentation

README.md

3.2 Chapter 1: Architecture

chapters/01_architecture.md

3.3 Chapter 2: Address Decode and Arbitration

chapters/02_address_and_arbitration.md

3.4 Chapter 3: RTL Generator

chapters/03_rtl_generator.md

3.5 Quick Navigation

3.5.1 For New Users

1. Start with README.md for overview and quick start
2. Read 01_architecture.md to understand the design
3. Study 02_address_and_arbitration.md for operational details
4. Reference 03_rtl_generator.md if you need a custom configuration

3.5.2 For Integration

- **Pre-generated variants:** See 01_architecture.md Section “Pre-Generated Variants”
- **Custom generation:** See 03_rtl_generator.md Section “Quick Start”
- **Address mapping:** See 02_address_and_arbitration.md Section “Address Decode”
- **Arbitration behavior:** See 02_address_and_arbitration.md Section “Arbitration”

3.5.3 Common Questions

All answered in README.md, section “Common Questions”.

3.6 Visual Assets

Every diagram in this specification is checked in as both source and rendered output:

- **Source Files:**
  - `assets/graphviz/*.gv` - Graphviz source diagrams
  - `assets/wavedrom/*.json` - WaveJSON timing diagrams
- **Rendered Files:**
  - `assets/png/*.png` - PNG format (for document generation)

3.6.1 Architecture Diagrams

1. **Lattice Architecture (2x4 Example)**
   - Source: assets/graphviz/apbx_xbar_architecture.gv
   - Rendered: assets/png/apbx_xbar_architecture.png
2. **Address Decode Flow**
   - Source: assets/graphviz/address_decode_flow.gv
   - Rendered: assets/png/address_decode_flow.png

3.6.2 Timing Diagrams

1. **Round-Robin Arbitration**
   - Source: assets/wavedrom/arbitration_round_robin.json
   - Rendered: assets/wavedrom/arbitration_round_robin.png

3.7 Component Overview

3.7.1 Key Features

- **Parametric MxN Configuration:** Any combination of M masters and N slaves (up to 16x16)
- **Automatic Address Decode:** 64KB per slave, simple offset-based routing
- **Round-Robin Arbitration:** Per-slave fair arbitration, no master starvation
- **Back-to-Back Transactions:** Accepted without master-side idle cycles, though they do not overlap inside the fabric (10 pclk cycles each for M = 1, 11 for arbitrated variants; one more than that variant's single-transfer latency — see HAS 5.2)
- **Grant Persistence:** Grant held through transaction completion
- **RTL Generation:** Python-based generator for custom configurations

3.7.2 Pre-Generated Variants

Module	M×N	Use Case
apbx_xbar_1to1	1×1	Passthrough, protocol conversion
apbx_xbar_2to1	2×1	Multi-master arbitration
apbx_xbar_1to4	1×4	Simple SoC peripheral bus
apbx_xbar_2to4	2×4	Typical SoC with CPU+DMA
apbx_xbar_2to2_mixed	2×2	Mixed APB4/APB5 (m0=APB4, m1=APB5; s0=APB5, s1=APB4)

3.7.3 Design Philosophy

Proven Building Blocks: - Built from production-tested `apb4_slave` and `apb4_master` modules - No new protocol logic - pure composition - Each component independently verified

Parametric Generation: - Generator creates any MxN configuration - Pre-generated common variants for fast integration - Custom variants generated on-demand

Clean Separation: - Master-side: APB slaves convert protocol → cmd/rsp - Internal: Arbitration + address decoding - Slave-side: APB masters convert cmd/rsp → protocol

---

3.8 Related Documentation

3.8.1 Companion Specifications

- **Lattice HAS** - Hardware Architecture Specification (high-level)

3.8.2 Project-Level

- **PRD.md:** ../../PRD.md - Complete product requirements document
- **CLAUDE.md:** ../../CLAUDE.md - AI assistant integration guide
- **README.md:** ../../README.md - Quick start guide

3.8.3 Test Infrastructure

- **Test Directory:** `../../dv/tests/` - CocoTB + pytest test suite
- **Test Results:** 100% passing (9/9) — all pre-generated variants plus the dedicated timing test; `apbx_xbar_2to2_mixed` additionally has a formal harness

3.8.4 RTL

- **Core Modules:** `../../rtl/apbx_xbar_*.sv` - Pre-generated crossbars
- **Wrappers:** `../../rtl/wrappers/` - testbench scaffolds (pclk/presetn only), not integration wrappers
- **Generator:** `../../bin/generate_xbars.py` - Python generator script

3.9 Version History

Version 1.0 (2025-10-25): - Initial specification release - Complete visual documentation (3 diagrams) - Generator documentation - All pre-generated variants verified (100% passing)

Version 1.1 (2026-10-12): - Refreshed against current RTL, generator, and DV - Updated verification status (9/9 tests passing, mixed-variant formal harness) - Added `--slave-size` CLI option to generator table - Updated line-of-code counts (single source of truth remains HAS ch05_performance/03_resources.md)

Last Updated: 2026-10-12 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 Product Requirements Document (PRD)

4.1 Lattice Generator

Version: 1.0 **Date:** 2025-10-19 **Status:** Production Ready **Owner:** RTL Design Sherpa Project **Parent Document:** `/PRD.md` **Codename:** lattice — directory `lattice-apbx-xbar` (fabric-gen-ip family with `bridge` and `delta`); RTL modules keep the `apbx_xbar_*` prefix

---

4.2 1. Executive Summary

The **Lattice** is a parametric generator for configurable MxN APB interconnect fabrics connecting multiple APB masters to multiple APB slaves. It is built from the proven `apb4_slave` and `apb4_master` modules: independent round-robin arbitration per slave, automatic address-based routing, and no new protocol logic.

4.2.1 1.1 Quick Stats

- **Modules:** 5 pre-generated variants + generator for custom sizes
- **Max Capacity:** Up to 16x16 (configurable)
- **Architecture:** apb4_slave → arbitration + decode → apb4_master
- **Status:** Production ready, all tests passing
- **Generator:** Python-based parametric code generation

4.2.2 1.2 Project Goals

- **Primary:** Provide production-quality APB interconnect for SoC integration
- **Secondary:** Demonstrate parametric RTL generation best practices
- **Tertiary:** Support both fixed and dynamically-generated variants

---

4.3 2. Key Design Principles

4.3.1 2.1 Architecture Philosophy

Proven Building Blocks: - Uses `apb4_slave.sv` and `apb4_master.sv` from rtl/amba/apb4 - Each module independently tested and production-proven - Crossbar = composition of proven components

Parametric Generation: - Generator creates any MxN configuration - Pre-generated common variants (1to1, 2to1, 1to4, 2to4, 2to2_mixed) - Custom variants generated on-demand

Clean Separation: - Master-side: APB slaves convert protocol to cmd/rsp - Internal: Arbitration + address decoding - Slave-side: APB masters convert cmd/rsp back to protocol

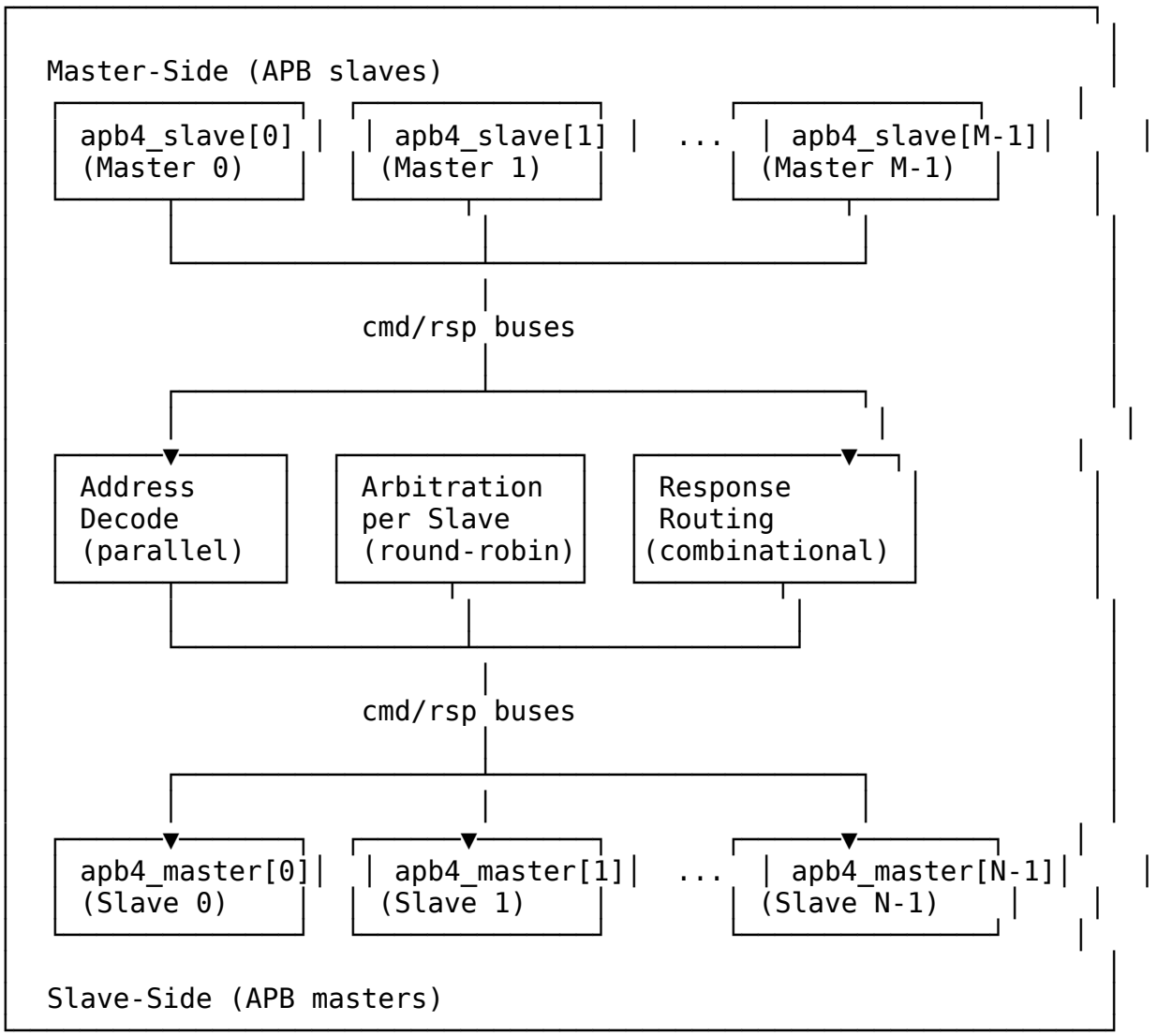
4.3.2 2.2 Design Trade-offs

Feature	Choice	Rationale
Arbitration	Round-robin per slave	Fair, simple, predictable
Address Map	64KB per slave	Sufficient for most peripherals
Grant Persistence	Hold through response	One transaction per grant; mask rotates after
Address Decode	Parallel decode	Low latency, simple logic

4.4 3. Architecture Overview

4.4.1 3.1 Top-Level Block Diagram

Lattice (M masters × N slaves)



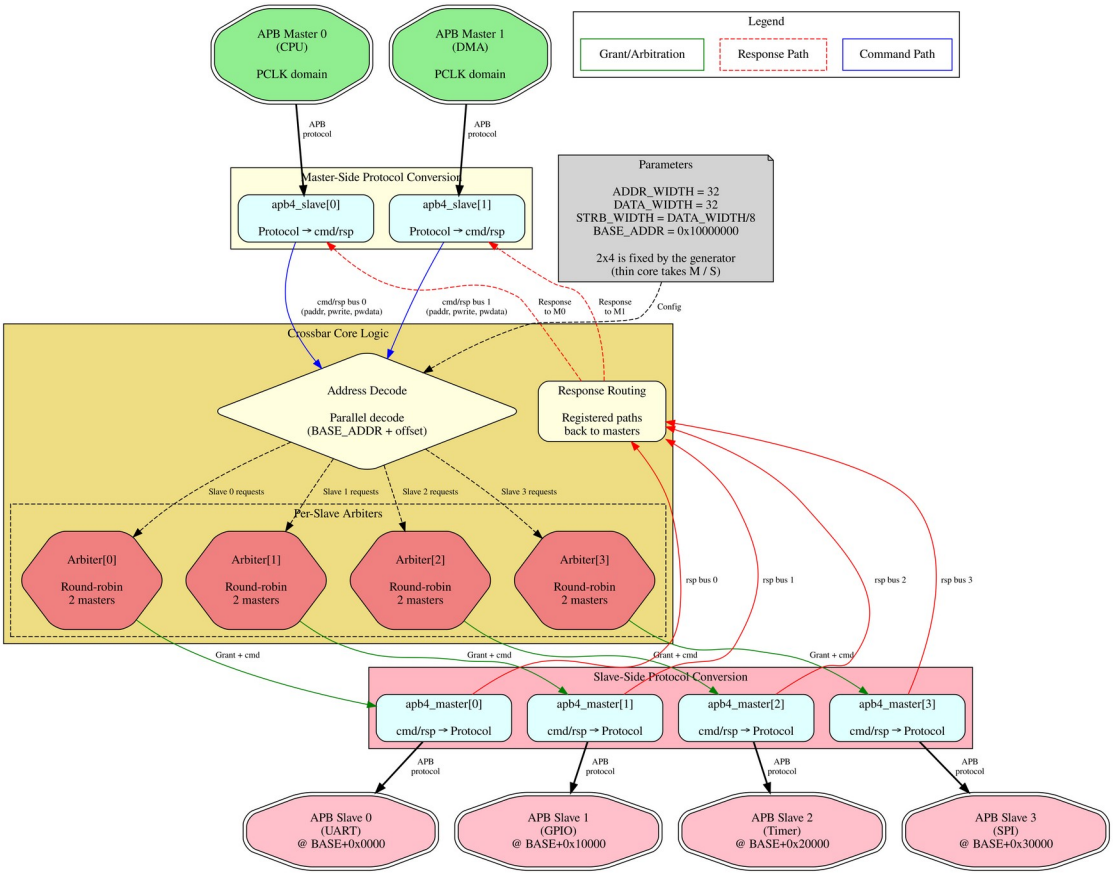


Figure: Lattice top-level architecture showing 2 masters connected to 4 slaves. *Source: docs/lattice_mas/assets/graphviz/apbx_xbar_architecture.gv* | *SVG*

4.4.2 3.2 Address Mapping

```
BASE_ADDR + 0x0000_0000 → 0x0000_FFFF : Slave 0 (64KB)
BASE_ADDR + 0x0001_0000 → 0x0001_FFFF : Slave 1 (64KB)
BASE_ADDR + 0x0002_0000 → 0x0002_FFFF : Slave 2 (64KB)
BASE_ADDR + 0x0003_0000 → 0x0003_FFFF : Slave 3 (64KB)
...
BASE_ADDR + 0x000F_0000 → 0x000F_FFFF : Slave 15 (64KB)
```

Default BASE_ADDR: 0x1000_0000

Address Decode Example:

The crossbar extracts the slave index from the upper bits of the address offset:

```
slave_index = (address - BASE_ADDR) >> 16  // Divide by 64KB (0x10000)
```

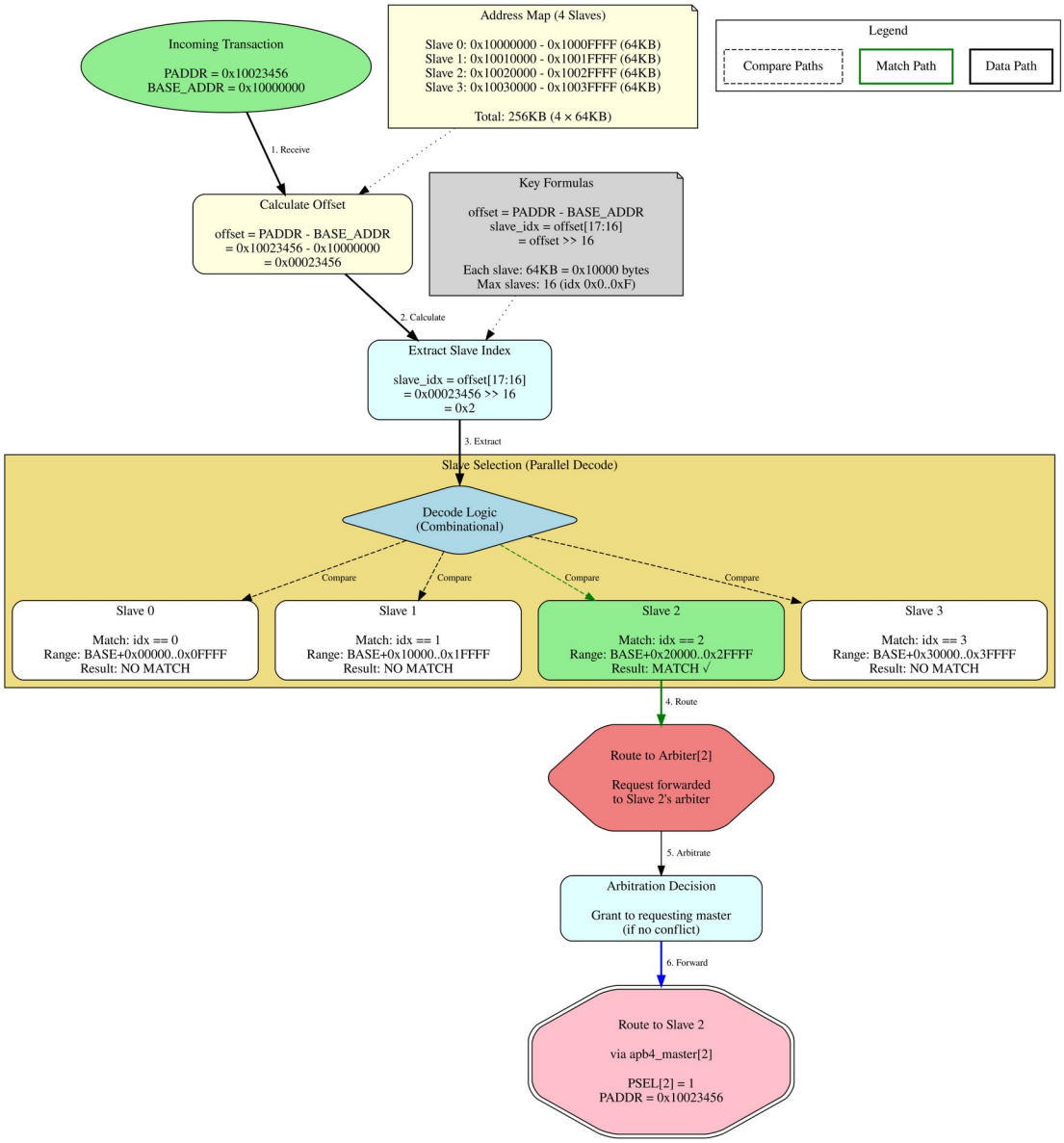


Figure: Address decode flow showing how address 0x10023456 routes to Slave 2. *Source: docs/lattice_mas/assets/graphviz/address_decode_flow.gv* | *SVG*

4.4.3 3.3 Arbitration Strategy

Per-Slave Round-Robin: - Each slave has independent arbiter - Master priority rotates after each grant - Grant held from command acceptance through response completion - No master can starve another master

Example:

```
Slave 0 accessed by M0, M1, M0 → Next grant goes to M1
Slave 1 accessed by M1, M1, M0 → Next grant goes to M1
```

```
(the winner cannot immediately re-win: after an M0 grant the
 mask leaves only M1 eligible)
```

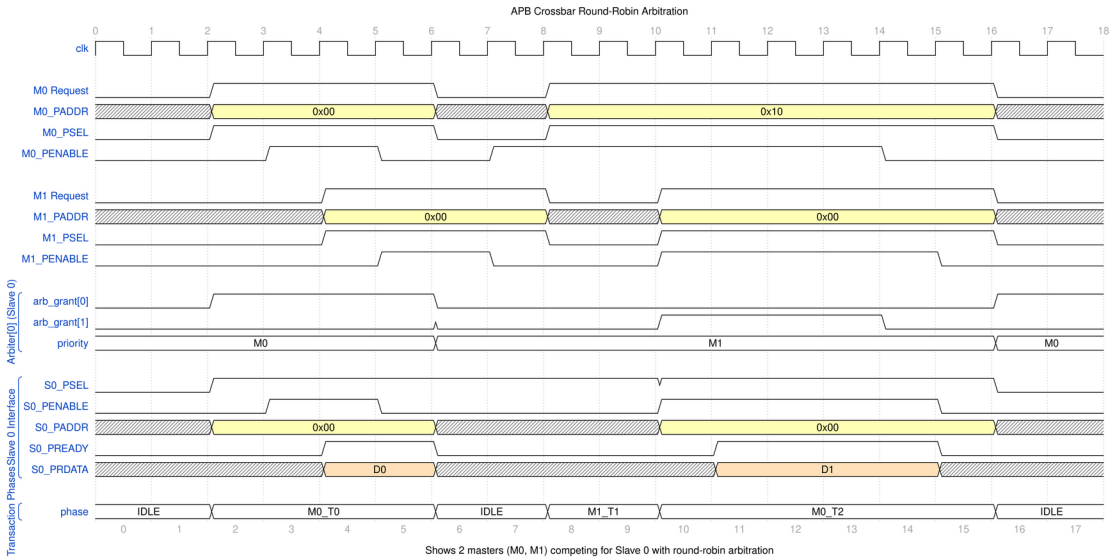


Figure: Round-robin arbitration timing showing 2 masters competing for Slave 0. Master priority rotates after each grant to ensure fair access. *Source: docs/lattice_mas/assets/wavedrom/arbitration_round_robin.json*

4.5 4. Available Variants

4.5.1 4.1 Pre-Generated Modules

Module	Masters	Slaves	Use Case
apbx_xbar_1to1	1	1	Protocol conversion, testing
apbx_xbar_2to1	2	1	Multi-master arbitration
apbx_xbar_1to4	1	4	Address decode, simple SoC
apbx_xbar_2to4	2	4	Full crossbar, typical SoC
apbx_xbar_2to2_mixed	2	2	Mixed APB4/APB5 (s0 is APB5)

LOC figures live in HAS ch05_performance/03_resources.md, which is the single source of truth for them. A duplicate column here disagreed with that table by up to 33%.

4.5.2 4.2 Wrapper Modules

These are TESTBENCH SCAFFOLDS, not integration building blocks. Each exposes `pclk` and `presetn` and nothing else — every APB signal is an internal net driven by the cocotb testbench. They exist because the four legacy tests drive a wrapper rather than the bare crossbar; a design cannot connect anything to one.

Wrapper	Configuration	Purpose
`apbx_xbar_1to1_wrap`	1×1	Test scaffold for `test_apbx_xbar_1to1.py`
`apbx_xbar_2to1_wrap`	2×1	Test scaffold for `test_apbx_xbar_2to1.py`
`apbx_xbar_1to4_wrap`	1×4	Test scaffold for `test_apbx_xbar_1to4.py`
`apbx_xbar_2to4_wrap`	2×4	Test scaffold for `test_apbx_xbar_2t`

Wrapper	Configuration	Purpose
		`o4.py`

To integrate, instantiate the crossbar itself. `apbx_xbar_2to2_mixed` has no wrapper precisely because its test drives the DUT directly, which is the better pattern.

4.6 5. Functional Requirements

4.6.1 FR-1: Arbitrary MxN Configuration

Priority: P0 (Critical) **Status:** Implemented and verified

Description: Generate crossbar for any M masters and N slaves (up to 16x16)

Verification: Generator creates valid RTL for all tested configurations

4.6.2 FR-2: Round-Robin Arbitration

Priority: P0 (Critical) **Status:** Implemented and verified

Description: Fair per-slave arbitration with rotating master priority

Verification: Arbitration stress tests (130+ transactions for 2to1)

4.6.3 FR-3: Address-Based Routing

Priority: P0 (Critical) **Status:** Implemented and verified

Description: Automatic slave selection based on address ranges

Verification: Address decode validation (200+ transactions for 1to4)

4.6.4 FR-4: Back-to-Back Transactions

Priority: P1 (High) **Status:** Implemented and verified

Description: Consecutive transactions are accepted with no master-side idle cycles required. They do NOT overlap inside the fabric — `apb4_slave` is one-command-at-a-time. Sustained cadence is 10 pclk cycles PREADY-to-PREADY for a single-master variant and 11 for an arbitrated one, in each case one more than that variant's SETUP-to-PREADY latency of 9 or 10 (HAS 5.1/5.2).

Verification: Performance tests show consecutive transactions

4.6.5 FR-5: Configurable Address Map

Priority: P1 (High) **Status:** Implemented and verified

Description: BASE_ADDR parameter sets base of address map

Verification: Tests with multiple BASE_ADDR values

4.7 6. Interface Specifications

4.7.1 6.1 Master-Side Interface (APB Slave)

Per-master APB slave interface:

```
input  logic                    m<i>_apb_PSEL
input  logic                    m<i>_apb_PENABLE
input  logic [ADDR_WIDTH-1:0]   m<i>_apb_PADDR
input  logic                    m<i>_apb_PWRITE
input  logic [DATA_WIDTH-1:0]   m<i>_apb_PWDATA
input  logic [STRB_WIDTH-1:0]   m<i>_apb_PSTRB
input  logic [2:0]              m<i>_apb_PPROT
output logic [DATA_WIDTH-1:0]   m<i>_apb_PRDATA
output logic                    m<i>_apb_PSLVERR
output logic                    m<i>_apb_PREADY
```

4.7.2 6.2 Slave-Side Interface (APB Master)

Per-slave APB master interface:

```
output logic                    s<j>_apb_PSEL
output logic                    s<j>_apb_PENABLE
output logic [ADDR_WIDTH-1:0]   s<j>_apb_PADDR
output logic                    s<j>_apb_PWRITE
output logic [DATA_WIDTH-1:0]   s<j>_apb_PWDATA
output logic [STRB_WIDTH-1:0]   s<j>_apb_PSTRB
output logic [2:0]              s<j>_apb_PPROT
input  logic [DATA_WIDTH-1:0]   s<j>_apb_PRDATA
input  logic                    s<j>_apb_PSLVERR
input  logic                    s<j>_apb_PREADY
```

4.8 7. Parameter Configuration

Parameter	Type	Default	Range	Description
ADDR_WIDTH	int	32	18-64 for multi-slave variants (the decode part-select needs bit 17); 1-64 for 1to1/2to1	Address bus width
DATA_WIDTH	int	32	8,16,32,64	Data bus width
STRB_WIDTH	int	DATA_WIDTH/8	-	Strobe width (auto-calc)
BASE_ADDR	logic [ADDR_WIDTH-1:0]	0x10000000	Any except the top S x 64KB, where BASE_ADDR + span wraps and the range check can never pass	Base address for slave map. Inert on 1to1/2to1 (no decode)

4.9 8. Generator Usage

4.9.1 8.1 Pre-Generated Variants

Use existing modules directly:

```
# Available in
projects/components/fabric-gen-ip/lattice-apbx-xbar/rtl/
apbx_xbar_1to1.sv
apbx_xbar_2to1.sv
apbx_xbar_1to4.sv
apbx_xbar_2to4.sv
apbx_xbar_2to2_mixed.sv
```

4.9.2 8.2 Custom Generation

Generate custom MxN crossbar:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py --masters 3 --slaves 6
python generate_xbars.py --masters 4 --slaves 8 --base-addr 0x80000000
```

4.9.3 8.3 Generator Options

```
python generate_xbars.py --help

Options:
  --masters M       Number of masters (1-16)
  --slaves N        Number of slaves (1-16)
  --base-addr ADDR  Base address (default: 0x10000000)
```

4.10 9. Performance Characteristics

4.10.1 9.1 Latency

Path	M = 1	M > 1	Notes
Command path	6	7	master SETUP (1) + forward path (5, or 6 with a registered arbiter grant)
Response path	3	3	arbitration is on the command side only
Total	**9**	**10**	SETUP->PREADY, uncontended, zero-wait slave (8 / 9 ACCESS->PREADY)

M = 1 is `1to1` and `1to4`, which the generator emits with no arbiter at all. M > 1 is `2to1`, `2to4` and `2to2_mixed` — including the 2x4 this document calls the typical SoC case. This table gave only the M = 1 figures, unlabelled, until 2026-08-30.

APB's 2-cycle minimum applies to a directly-attached slave, not through this fabric: the crossbar converts APB→cmd/rsp→APB across registered skid buffers in both directions. See HAS 5.2 for the cycle-by-cycle breakdown and the measurement.

4.10.2 9.2 Throughput

- **Back-to-back transactions:** Supported, but not overlapped — `apb4_slave` is a one-command-at-a-time FSM, so the next command is captured only after the previous transaction completes
- **Maximum rate:** ~1 transaction per 10 pclk cycles (1to1, 1to4) or 11 (2to1, 2to4, 2to2_mixed) per master (measured back-to-back at an always-ready slave)

4.10.3 9.3 Resource Utilization (Estimated)

See HAS ch05_performance/03_resources.md — that table is the single source of truth. An earlier duplicate here quoted figures 2-3x smaller in every cell; both were unsourced hand estimates, so the duplicate is withdrawn rather than reconciled.

4.11 10. Verification Status

4.11.1 10.1 Test Coverage

Test	Transactions	Status	Coverage
test_apbx_xbar_1to1	100+	Pass	Basic connectivity
test_apbx_xbar_2to1	130+	Pass	Arbitration stress
test_apbx_xbar_1to4	200+	Pass	Address decode
test_apbx_xbar_2to4	350+	Pass	Full crossbar stress
test_apbx_xbar_2to2_mixed	8 directed	Pass	All four APB4/APB5 pairings, sideband gating, decode miss, structural pin check
test_apbx_xbar_timing	2 variants	Pass	Published latency/cadence figures, both variant classes

Overall: 100% passing (9/9), >750 transactions plus the directed mixed-version and timing checks. `2to2_mixed` also has a formal harness (`formal/apbx_xbar/apbx_xbar_2to2_mixed/`).

4.11.2 10.2 Test Methodology

Test Structure: - CocoTB framework - Random transaction generation - Variable delay profiles - Address range validation - Arbitration fairness checks

Verification Points: - Correct address routing - Round-robin arbitration - Response integrity - Back-to-back transactions - Error propagation

---

4.12 11. Integration Guide

4.12.1 11.1 Simple Integration (1to4)

```
apbx_xbar_1to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)
) u_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // Master interface
    .m0_apb_PSEL(cpu_psel),
    .m0_apb_PENABLE(cpu_penable),
    .m0_apb_PADDR(cpu_paddr),
    .m0_apb_PWRITE(cpu_pwrite),
    .m0_apb_PWDATA(cpu_pwdata),
    .m0_apb_PSTRB(cpu_pstrb),
    .m0_apb_PPROT(cpu_pprot),
    .m0_apb_PRDATA(cpu_prdata),
    .m0_apb_PSLVERR(cpu_pslverr),
    .m0_apb_PREADY(cpu_pready),

    // Slave 0: UART (0x1000_0000 - 0x1000_FFFF)
    .s0_apb_PSEL(uart_psel),
    .s0_apb_PENABLE(uart_penable),
    // ... (similar connections)

    // Slave 1: GPIO (0x1001_0000 - 0x1001_FFFF)
    .s1_apb_PSEL(gpio_psel),
    // ... (similar connections)

    // Slave 2: Timer (0x1002_0000 - 0x1002_FFFF)
    // Slave 3: SPI (0x1003_0000 - 0x1003_FFFF)
);
```

4.12.2 11.2 Multi-Master Integration (2to4)

```
apbx_xbar_2to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)
) u_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // Master 0: CPU
```

```
    .m0_apb_PSEL(cpu_psel),
    // ... (full interface)

    // Master 1: DMA
    .m1_apb_PSEL(dma_psel),
    // ... (full interface)

    // Slaves 0-3: Peripherals
    // ... (slave connections)
);
```

4.13 12. Design Constraints

4.13.1 12.1 Limitations

Constraint	Value	Rationale
Max masters	16	Generator limit (configurable)
Max slaves	16	Generator limit (configurable)
Slave region size	64KB in every shipped variant	Baked in at generation (`--slave-size`, default 4KB); not overridable at elaboration
No slave disable	N/A	All slaves always active
No timeout	N/A	Assumes slaves always respond

4.13.2 12.2 Assumptions

1. **Slave responses:** All slaves respond eventually (no timeout handling)
2. **Address ranges:** Slaves occupy 64KB regions starting at BASE_ADDR
3. **APB compliance:** All masters and slaves follow APB protocol
4. **Single clock domain:** All signals synchronous to pclk

4.14 13. Future Enhancements

4.14.1 13.1 Potential Features

- **Configurable slave region sizes:** Not just 64KB
- **Slave enable/disable:** Dynamic slave activation
- **Timeout detection:** Watchdog for hung slaves
- **Transaction ordering:** Optional in-order completion
- **Priority arbitration:** Weighted instead of round-robin

4.14.2 13.2 Generator Improvements

- **HDL output formats:** Support Verilog, VHDL, Chisel
- **Automated testing:** Generate tests alongside RTL
- **Documentation generation:** Auto-generate integration guides
- **Synthesis scripts:** Generate vendor-specific scripts

4.15 14. References

4.15.1 14.1 Internal Documentation

- `README.md` - Quick start guide
- `CLAUDE.md` - AI assistant guidelines
- `bin/generate_xbars.py` - Generator script
- `dv/tests/` - Test implementations

4.15.2 14.2 External Standards

- **AMBA APB Protocol v2.0** - ARM IHI 0024C
- **APB4 Extensions** - AMBA 5 specification

4.15.3 14.3 Related Components

- `rtl/amba/apb4/apb4_slave.sv` - Master-side building block
- `rtl/amba/apb4/apb4_master.sv` - Slave-side building block
- `projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/apbx_xbar_generator.py` - Main generator

Document Version: 1.0 **Last Review:** 2025-10-19 **Next Review:** 2026-01-01 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Claude Code Guide: Lattice Component

Version: 1.0 **Last Updated:** 2025-10-19 **Purpose:** AI-specific guidance for working with Lattice component

Codename: lattice · **Directory:** `lattice-apbx-xbar` (fabric-gen-ip family with `bridge` and `delta`); RTL modules keep the `apbx_xbar_*` prefix

---

5.1 Quick Context

What: Lattice Generator - Parametric APB interconnect for connecting M masters to N slaves
Status: Production Ready (all tests passing) **Your Role:** Help users generate, integrate, and customize APB crossbars

Complete Specification:
`projects/components/fabric-gen-ip/lattice-apbx-xbar/PRD.md` ← **Always reference this for technical details**

5.2 Critical Rules for This Component

5.2.1 Rule #0: This is a GENERATOR, Not Just Modules

IMPORTANT: Lattice is both: 1. **Pre-generated modules** (1to1, 2to1, 1to4, 2to4, 2to2_mixed) in `rtl/` 2. **Python generator** (`bin/generate_xbars.py`) for custom configurations

When users ask for crossbar: - Check if pre-generated variant exists first - Only suggest generation if custom size needed

Decision Tree:

```
User needs crossbar?
├─ 1x1, 2x1, 1x4, 2x4? → Use pre-generated module
├─ Mixed APB4/APB5 2x2? → Use apbx_xbar_2to2_mixed
└─ Custom MxN? → Run generator script
```

5.2.2 Rule #1: Address Map is Fixed Per-Slave

Each slave occupies 64KB region:

```
Slave 0: BASE_ADDR + 0x0000_0000 → 0x0000_FFFF
Slave 1: BASE_ADDR + 0x0001_0000 → 0x0001_FFFF
Slave 2: BASE_ADDR + 0x0002_0000 → 0x0002_FFFF
...
```

The window size is not a module parameter, but it IS a generator option. Every *shipped* variant uses 64KB windows because `bin/generate_xbars.py` passes `slave_size=0x10000`. The generator's own default is 4KB, and `apbx_xbar_generator.py --slave-size` accepts any power of 2 >= 256 bytes. Once generated the size is baked into the decode part-select, so it cannot be overridden at elaboration — only `BASE_ADDR` can, and only on the decoding variants.

If user asks for different sizes:

```
WRONG: "Let me modify the generator to support custom sizes per slave"

CORRECT: "The shipped variants use 64KB per slave. You can:
1. Use BASE_ADDR to shift the whole map (decoding variants only)
2. For a different window, regenerate with --slave-size (power of 2,
>= 256B)
3. Or use multiple crossbars with different BASE_ADDR values"
```

5.2.3 Rule #2: Know the Pre-Generated Variants

Available in `rtl/` directory:

Module	M×N	Use Case	When to Recommend
apbx_xbar_1to1	1×1	Passthrough	Protocol conversion, testing
apbx_xbar_2to1	2×1	Arbitration	Multi-master to single peripheral
apbx_xbar_1to4	1×4	Decode	Single CPU to multiple peripherals
apbx_xbar_2to4	2×4	Full crossbar	CPU + DMA to peripherals
apbx_xbar_2to2_mixed	2×2	Mixed APB4/APB5	APB5 peripheral alongside APB4

Always suggest pre-generated first!

5.2.4 Rule #3: Generator Syntax

Generate custom crossbar:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py --masters 3 --slaves 6
```

Options:

```
--masters M         Number of masters (1-16)
--slaves N          Number of slaves (1-16)
--base-addr ADDR    Base address (default: 0x10000000)
```

Example:

```
# Generate 3x8 crossbar with custom base
python generate_xbars.py --masters 3 --slaves 8 --base-addr 0x80000000

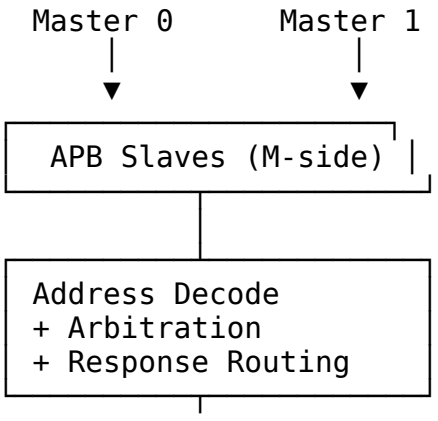
# Generate a 5x5 variant
python generate_xbars.py --masters 5 --slaves 5
```

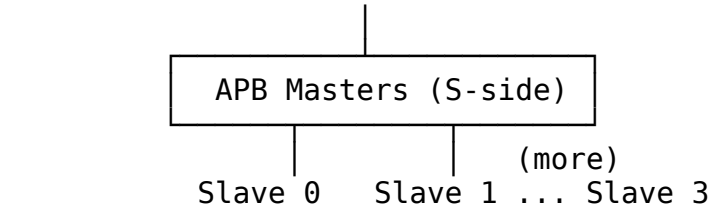
5.3 Architecture Quick Reference

5.3.1 Module Organization

```
projects/components/fabric-gen-ip/lattice-apbx-xbar/
├── rtl/
│   ├── apbx_xbar_1to1.sv        Core crossbar modules
│   ├── apbx_xbar_2to1.sv
│   ├── apbx_xbar_1to4.sv
│   ├── apbx_xbar_2to4.sv
│   ├── apbx_xbar_2to2_mixed.sv  (m0=APB4, m1=APB5, s0=APB5, s1=APB4)
│   └── wrappers/                testbench scaffolds, not integration
wrappers
│       ├── apbx_xbar_1to1_wrap.sv
│       ├── apbx_xbar_2to1_wrap.sv
│       ├── apbx_xbar_1to4_wrap.sv
│       └── apbx_xbar_2to4_wrap.sv
├── bin/
│   └── generate_xbars.py       Generator script
├── dv/
│   └── tests/
│       ├── test_apbx_xbar_1to1.py
│       ├── test_apbx_xbar_2to1.py
│       ├── test_apbx_xbar_1to4.py
│       ├── test_apbx_xbar_2to4.py
│       ├── test_apbx_xbar_2to2_mixed.py
│       ├── test_apbx_xbar_timing.py     published latency/cadence
figures
│       ├── test_apbx_xbar_parity.py     generates a parity-on all-
APB5 2x2 (APB5 *CHK, PWAKEUP)
│       └── GTKW/                Waveform configs
├── PRD.md                       Complete specification
├── CLAUDE.md                    This file
└── README.md                    Quick start guide
```

5.3.2 Block Diagram (2×4 Example)




APB Masters (S-side)
(more)
Slave 0
Slave 1 ... Slave 3

5.4 Common User Questions and Responses

5.4.1 Q: "How do I connect a CPU to 4 peripherals?"

A: Use the pre-generated 1to4 crossbar:

```
apbx_xbar_1to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)  // Start of peripheral space
) u_peripheral_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // CPU connection (master 0)
    .m0_apb_PSEL(cpu_psel),
    .m0_apb_PENABLE(cpu_penable),
    .m0_apb_PADDR(cpu_paddr),
    .m0_apb_PWRITE(cpu_pwrite),
    .m0_apb_PWDATA(cpu_pwdata),
    .m0_apb_PSTRB(cpu_pstrb),
    .m0_apb_PPROT(cpu_pprot),
    .m0_apb_PRDATA(cpu_prdata),
    .m0_apb_PSLVERR(cpu_pslverr),
    .m0_apb_PREADY(cpu_pready),

    // Peripheral 0: UART @ 0x1000_0000
    .s0_apb_PSEL(uart_psel),
    .s0_apb_PENABLE(uart_penable),
    // ... (full interface)

    // Peripheral 1: GPIO @ 0x1001_0000
    // Peripheral 2: Timer @ 0x1002_0000
    // Peripheral 3: SPI @ 0x1003_0000
);
```

Address map: - UART: 0x1000_0000 - 0x1000_FFFF - GPIO: 0x1001_0000 - 0x1001_FFFF - Timer: 0x1002_0000 - 0x1002_FFFF - SPI: 0x1003_0000 - 0x1003_FFFF

See: PRD.md Section 11.1

5.4.2 Q: "I need CPU and DMA to access peripherals. Which crossbar?"

A: Use the 2to4 crossbar:

```
apbx_xbar_2to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
```

```
    .BASE_ADDR(32'h1000_0000)
) u_soc_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // Master 0: CPU
    .m0_apb_PSEL(cpu_psel),
    // ... (full interface)

    // Master 1: DMA
    .m1_apb_PSEL(dma_psel),
    // ... (full interface)

    // Slaves 0-3: Peripherals
    // ... (slave connections)
);
```

Key feature: Round-robin arbitration per slave ensures fair access between CPU and DMA.

See: PRD.md Section 11.2

5.4.3 Q: "What if I need 3 masters and 8 slaves?"

A: Generate custom crossbar:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py --masters 3 --slaves 8   # writes
apbx_xbar_3to8.sv next to the script

# Check generated file, then move it into rtl/
ls -lh apbx_xbar_3to8.sv
mv apbx_xbar_3to8.sv ../rtl/
```

Then instantiate like any other crossbar:

```
apbx_xbar_3to8 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)
) u_custom_xbar (
    // ... connections for 3 masters and 8 slaves
);
```

See: PRD.md Section 8

5.4.4 Q: "How does arbitration work?"

A: Per-slave round-robin:

Each slave has its own arbiter that rotates master priority:

```
Example with 2 masters accessing Slave 0:

Transaction 1: M0 requests → M0 granted
Transaction 2: M0 and M1 request → M1 granted (rotated)
Transaction 3: M1 requests → M1 granted
Transaction 4: M0 and M1 request → M0 granted (rotated)
```

Key points: - **Independent per slave:** Each slave arbitrates independently - **Fair:** No master can starve another - **Grant persistence:** Once granted, master holds slave until response completes - **Back-to-back:** supported, but not overlapped — 10 pclk cycles per transfer for 1to1/1to4, 11 for the arbitrated variants (2to1, 2to4, 2to2_mixed), PREADY-to-PREADY (see HAS 5.1/5.2)

See: PRD.md Section 3.3

5.4.5 Q: "Can I change the address map?"

A: BASE_ADDR parameter shifts entire map, but per-slave size is fixed:

```
// Default: 0x1000_0000
apbx_xbar_1to4 #(.BASE_ADDR(32'h1000_0000)) u_xbar1 (...);
// Slaves at: 0x1000_0000, 0x1001_0000, 0x1002_0000, 0x1003_0000

// Shifted: 0x8000_0000
apbx_xbar_1to4 #(.BASE_ADDR(32'h8000_0000)) u_xbar2 (...);
// Slaves at: 0x8000_0000, 0x8001_0000, 0x8002_0000, 0x8003_0000
```

Limitation: Each slave always occupies 64KB (0x10000 bytes)

Workaround for different sizes: 1. Use multiple crossbars with different BASE_ADDR 2. Modify generator's `addr_offset` calculation 3. Use address masking in slaves

See: PRD.md Section 3.2

5.4.6 Q: "How do I run tests?"

A: Use the area Makefile (never bare pytest):

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/dv/tests/

# Run specific test
make run-apbx_xbar_1to4-gate

# Run all tests -- clean-all first for a run you intend to trust
make clean-all && make run-all-gate

# Run with waveforms (WAVES=1 via the target, not --vcd)
```

```
make run-apbx_xbar_2to4-gate-waves
# create_view_cmd() writes the gtkwave command beside the log
```

Test coverage: - **test_apbx_xbar_1to1**: 100+ transactions (passthrough) - **test_apbx_xbar_2to1**: 130+ transactions (arbitration) - **test_apbx_xbar_1to4**: 200+ transactions (address decode) - **test_apbx_xbar_2to4**: 350+ transactions (full stress) - **test_apbx_xbar_2to2_mixed**: 8 directed — all four APB4/APB5 pairings, sideband gating, decode miss, structural pin check; plus `_pwakeup`: PWAKEUP forwarded requester -> completer (IHI 0024E 3.7.2) - **test_apbx_xbar_parity**: no shipped variant enables parity, so this generates one into its sim_build and checks odd `*CHK` parity (IHI 0024E ch. 5), error injection, and PWAKEUP forwarding - **test_apbx_xbar_timing**: both variant classes — asserts the published latency, cadence and per-path breakdown so the numbers cannot drift

All tests passing (22/22 at func, 2026-10-12) — and `2to2_mixed` additionally has a formal harness at `formal/apbx_xbar/apbx_xbar_2to2_mixed/`.

See: `PRD.md` Section 10

5.5 Integration Patterns

5.5.1 Pattern 1: Simple Peripheral Interconnect

```
// CPU to 4 peripherals
apbx_xbar_1to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)
) u_periph_xbar (
    .pclk(sys_clk),
    .presetn(sys_rst_n),

    // CPU side
    .m0_apb_PSEL(cpu_apb_psel),
    .m0_apb_PENABLE(cpu_apb_penable),
    .m0_apb_PADDR(cpu_apb_paddr),
    .m0_apb_PWRITE(cpu_apb_pwrite),
    .m0_apb_PWDATA(cpu_apb_pwdata),
    .m0_apb_PSTRB(cpu_apb_pstrb),
    .m0_apb_PPROT(cpu_apb_pprot),
    .m0_apb_PRDATA(cpu_apb_prdata),
    .m0_apb_PSLVERR(cpu_apb_pslverr),
    .m0_apb_PREADY(cpu_apb_pready),

    // Peripherals
    .s0_apb_PSEL(uart_psel), /* ... full interface ... */
    .s1_apb_PSEL(gpio_psel), /* ... */
    .s2_apb_PSEL(timer_psel), /* ... */
    .s3_apb_PSEL(spi_psel) /* ... */
);
```

5.5.2 Pattern 2: Multi-Master System

```
// CPU + DMA to peripherals
apbx_xbar_2to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h4000_0000)
) u_soc_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // Master 0: CPU
    .m0_apb_PSEL(cpu_psel),
    /* ... full CPU interface ... */

    // Master 1: DMA Controller
```

```
    .m1_apb_PSEL(dma_psel),
    /* ... full DMA interface ... */

    // Slave 0-3: Memory-mapped peripherals
    .s0_apb_PSEL(mem_ctrl_psel), /* ... */
    .s1_apb_PSEL(uart_psel), /* ... */
    .s2_apb_PSEL(i2c_psel), /* ... */
    .s3_apb_PSEL(adc_psel) /* ... */
);
```

5.5.3 Pattern 3: Hierarchical Interconnect

```
// Top-level crossbar
apbx_xbar_1to4 u_top_xbar (
    // .m0_apb_PSEL(cpu_apb_PSEL), .m0_apb_PENABLE(...), ...  (elided)
    .s0_apb_PSEL(periph_bus0_PSEL), /* ...rest of the port... */  //
To sub-crossbar 0
    .s1_apb_PSEL(periph_bus1_PSEL), /* ...rest of the port... */  //
To sub-crossbar 1
    .s2_apb_PSEL(mem_ctrl_PSEL), /* ...rest of the port... */     //
Direct to memory controller
    .s3_apb_PSEL(dma_ctrl_PSEL), /* ...rest of the port... */      //
Direct to DMA
);

// Sub-crossbar 0 for low-speed peripherals
apbx_xbar_1to4 u_periph_xbar0 (
    .m0_apb_PSEL(periph_bus0_PSEL), /* ...rest of the port... */
    .s0_apb_PSEL(uart0_PSEL), /* ...rest of the port... */
    .s1_apb_PSEL(gpio0_PSEL), /* ...rest of the port... */
    .s2_apb_PSEL(i2c0_PSEL), /* ...rest of the port... */
    .s3_apb_PSEL(spi0_PSEL), /* ...rest of the port... */
);

// Sub-crossbar 1 for high-speed peripherals
apbx_xbar_1to4 u_periph_xbar1 (
    .m0_apb_PSEL(periph_bus1_PSEL), /* ...rest of the port... */
    .s0_apb_PSEL(uart1_PSEL), /* ...rest of the port... */
    .s1_apb_PSEL(timer_PSEL), /* ...rest of the port... */
    .s2_apb_PSEL(pwm_PSEL), /* ...rest of the port... */
    .s3_apb_PSEL(adc_PSEL), /* ...rest of the port... */
);
```

5.6 Anti-Patterns to Catch

5.6.1 Anti-Pattern 1: Generating When Pre-Generated Exists

```
WRONG:
User: "I need a 2×4 crossbar"
You: "Let me generate that for you..."
python generate_xbars.py --masters 2 --slaves 4

CORRECTED:
"Use the pre-generated apbx_xbar_2to4.sv in the rtl/ directory.
No generation needed!"
```

5.6.2 Anti-Pattern 2: Assuming Custom Per-Slave Sizes

```
WRONG:
User: "Can I make slave 0 256KB and slave 1 4KB?"
You: "Sure, let me modify the parameters..."

CORRECTED:
"The shipped variants use 64KB per slave. For a different window:
1. Regenerate with --slave-size (power of 2, >= 256B)
2. Or use multiple crossbars with different BASE_ADDR
3. Or implement address masking in slaves"
```

5.6.3 Anti-Pattern 3: Not Mentioning Address Map

```
WRONG:
User: "How do I integrate the crossbar?"
You: *Shows port connections only*

CORRECTED:
"Here's the integration with address map:
apbx_xbar_1to4 #(.BASE_ADDR(32'h1000_0000)) u_xbar (...);

Address map:
- Slave 0: 0x1000_0000 - 0x1000_FFFF
- Slave 1: 0x1001_0000 - 0x1001_FFFF
- Slave 2: 0x1002_0000 - 0x1002_FFFF
- Slave 3: 0x1003_0000 - 0x1003_FFFF"
```

5.6.4 Anti-Pattern 4: Forgetting About Wrappers

```
WRONG:
User: "I need a quick 10×10 crossbar"
You: "Run the generator with --masters 10 --slaves 10"

CORRECTED:
"We have pre-configured wrappers in rtl/wrappers/:
```

```
- apbx_xbar_1to1_wrap.sv, apbx_xbar_2to1_wrap.sv,
  apbx_xbar_1to4_wrap.sv, apbx_xbar_2to4_wrap.sv

Those four are TESTBENCH SCAFFOLDS, not integration wrappers: each has
only
pclk and presetn, with every APB signal internal to the cocotb bench,
so a
design cannot connect to one. Instantiate the crossbar directly. For
any
other MxS, generate one with bin/generate_xbars.py -- and there is no
m10_s10 wrapper in the tree."
```

5.7 Debugging Workflow

5.7.1 Issue: Address Not Routing Correctly

Check in order: 1. BASE_ADDR parameter set correctly? 2. Address within slave's 64KB region? 3. PSEL signal asserted by master? 4. All APB signals properly connected?

Calculate expected slave:

```
slave_index = (address - BASE_ADDR) >> 16  // Divide by 64KB
```

Debug commands:

```
cd dv/tests && make run-apbx_xbar_1to4-gate-waves
# gtkwave command is written beside the log by create_view_cmd()
```

5.7.2 Issue: Arbitration Not Fair

Symptoms: - One master dominates - Other masters starved

Check: 1. Arbiters instantiated per slave? 2. Round-robin logic correct? 3. Grant persistence working?

Verify with tests:

```
cd dv/tests && make run-apbx_xbar_2to1-gate   # Arbitration stress
test
```

5.7.3 Issue: Back-to-Back Transactions Stalling

Check: 1. Grant persistence enabled? 2. Slaves responding with PREADY? 3. No unintended pipeline bubbles?

View waveforms:

```
cd dv/tests && make run-apbx_xbar_2to4-gate-waves
# gtkwave command is written beside the log by create_view_cmd()
```

5.8 Quick Commands

```
# List available crossbars
ls projects/components/fabric-gen-ip/lattice-apbx-xbar/rtl/*.sv

# Generate custom crossbar
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py --masters 3 --slaves 6

# Run all tests
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/dv/tests
make clean-all && make run-all-gate

# Run specific test with waveforms
make run-apbx_xbar_2to4-gate-waves

# View waveforms: create_view_cmd() writes a ready-made command
# beside the log; no hand-built gtkwave path needed

# Check documentation
cat projects/components/fabric-gen-ip/lattice-apbx-xbar/PRD.md
cat projects/components/fabric-gen-ip/lattice-apbx-xbar/README.md
```

5.9 Remember

1. **Check pre-generated first** - Don't generate unnecessarily
2. **Address map matters** - Always mention BASE_ADDR + 64KB regions
3. **Fair arbitration** - Round-robin per slave
4. **Complete connections** - All APB signals must be wired
5. **Tests available** - 100% passing, comprehensive coverage
6. **Wrappers are TEST SCAFFOLDS** - `rtl/wrappers/` carries only pclk/presetn; instantiate the crossbar directly (see Anti-Pattern 4)
7. **Generator limits** - Up to 16×16 (configurable)

Version: 1.0 **Last Updated:** 2025-10-19 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

6 Document Information

6.1 Lattice Generator Architecture Specification

Property	Value
Document Title	Lattice Generator Architecture Specification
Version	1.0
Date	2026-10-12
Status	Released
Classification	Open Source - MIT License

6.2 Revision History

Version	Date	Author	Description
1.0	2026-10-12	RTL Design Sherpa	Initial GAS release — verified against `apbx_xbar_generator.py`, `generate_xbars.py`, and emitted fixtures in `rtl/`.
1.2	2026-10-13	seang	Rebrand: component codename **lattice**, directory `lattice-apbx-xbar`; book titles and document number (LATTICE-GAS-001) updated; RTL module names unchanged (`apbx_xbar_*`)

6.3 Document Purpose

This Generator Architecture Specification (GAS) documents the Lattice code generator. It is written for the engineer who must re-run the generator, add a new variant or option, or understand why a generated module looks the way it does.

For the generated hardware, see the **Lattice HAS** and **MAS**. For product requirements, see **PRD.md**.

6.4 Intended Audience

- RTL engineers extending the generator
- Verification engineers regenerating fixtures
- Integrators choosing pre-generated vs custom variants

6.5 Related Documents

- **Lattice HAS** - Hardware Architecture Specification
- **Lattice MAS** - Micro-Architecture Specification
- **PRD.md** - Product requirements
- **CLAUDE.md** - AI development guide

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

7 What the Generator Produces

The Lattice generator (`bin/apbx_xbar_generator.py`) emits a single-file SystemVerilog module for an M×N APB crossbar. The module is parameterized at elaboration time for address width, data width, and base address; the master/slave counts and address-map stride are fixed at generation time.

7.1 Output artifacts

Artifact	Description
`apbx_xbar_<M>to<N>.sv`	Core crossbar module.
`apbx_xbar_<M>to<N>_wrap.sv`	Testbench scaffold (only for the four legacy fixtures).
`apbx_xbar_<M>to<N>.f`	Filelist (when generated by an area flow, not by the generator itself).

The generator emits the final form: banner, SPDX header, `reset_defs.svh` include, module header, and complete internal logic. No post-processing is required.

7.2 Architecture in the generated module

The generated module contains:

1. **Master-side APB slave modules** — `apb4_slave` or `apb5_slave` per master, converting APB → cmd/rsp.
2. **cmd/rsp fabric** — address decode, per-slave round-robin arbitration, response routing.
3. **Slave-side APB master modules** — `apb4_master` or `apb5_master` per slave, converting cmd/rsp → APB.

The generator also supports mixed APB4/APB5 ports and optional APB5 parity.

8 When to Regenerate

Regenerate when any of the following change:

- `bin/apbx_xbar_generator.py`
- `bin/generate_xbars.py`
- The `apb4_slave` / `apb4_master` / `apb5_slave` / `apb5_master` boundary modules
- The `arbiter_round_robin` module

The standard regeneration workflow is:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin
python generate_xbars.py        # regenerates all standard variants
into rtl/
```

After regeneration, run the DV suite:

```
cd ../dv/tests
make clean-all && make run-all-gate
```

8.1 Pre-generated vs custom

Always check whether a pre-generated variant exists before generating a custom one. The pre-generated fixtures are the regression baseline; custom variants are for one-off sizes.

9 Generator Entry Points

9.1 Low-level generator

```
python3 apbx_xbar_generator.py --masters 2 --slaves 4 --output
apbx_xbar_2to4.sv
```

CLI flags are documented in the configuration chapter.

9.2 Convenience batch script

```
python3 generate_xbars.py              # all standard variants
python3 generate_xbars.py --masters 3 --slaves 6
python3 generate_xbars.py --masters 4 --slaves 8 --base-addr
0x80000000
```

9.3 Programmatic API

```
from apbx_xbar_generator import generate_apbx_xbar

code = generate_apbx_xbar(
    num_masters=2,
    num_slaves=4,
    base_addr=0x10000000,
    addr_width=32,
    data_width=32,
    output_file="apbx_xbar_2to4.sv",
    slave_size=0x10000,
    master_versions=['apb4', 'apb4'],
    slave_versions=['apb4', 'apb4'],
    name_suffix="",
    enable_parity=False,
)
```

The function returns the generated SystemVerilog as a string and also writes the file when `output_file` is provided.

---

10 CLI and Programmatic API

The generator is invoked either directly for a custom crossbar or through the batch script for the standard family.

10.1 apbx_xbar_generator.py

```
python3 apbx_xbar_generator.py --masters 2 --slaves 4 --output
apbx_xbar_2to4.sv
```

Flag	Short	Default	Description
`--masters`	`-m`	required	Number of master interfaces (1-16).
`--slaves`	`-s`	required	Number of slave interfaces (1-16).
`--base-addr`	`-b`	`0x10000000`	Base address of the slave address map.
`--slave-size`		`0x1000`	Address space per slave; must be a power of two and at least 256 bytes.
`--addr-width`	`-a`	32	APB PADDR width.
`--data-width`	`-d`	32	APB PWDATA/PRDATA width.
`--output`	`-o`	`apbx_xbar_<M>to<N>.sv`	Output filename.

The CLI does **not** expose per-port APB5 or parity knobs; those are controlled by the programmatic API and the batch script.

10.2 generate_xbars.py

```
python3 generate_xbars.py                # regenerate all standard
variants
python3 generate_xbars.py --masters 3 --slaves 6
python3 generate_xbars.py --masters 4 --slaves 8 --base-addr
0x80000000 --slave-size 0x10000
```

Flag	Short	Default	Description
`--masters`	`-m`	(all)	Masters for a custom variant.
`--slaves`	`-s`	(all)	Slaves for a custom variant.
`--base-addr`	`-b`	`0x10000000`	Base address for custom variant.
`--slave-size`		`0x10000`	Stride for custom variant (default matches the shipped family).

The batch script explicitly passes `slave_size=0x10000` to the generator so the shipped family uses 64KB windows. Omitting the flag would silently fall back to the generator's 4KB default, which is why the batch script always passes it.

10.3 Programmatic API

```
from apbx_xbar_generator import generate_apbx_xbar

code = generate_apbx_xbar(
    num_masters=2,
    num_slaves=4,
    base_addr=0x10000000,
    addr_width=32,
    data_width=32,
    output_file="apbx_xbar_2to4.sv",
    slave_size=0x10000,
    master_versions=['apb4', 'apb4'],
    slave_versions=['apb4', 'apb4'],
    name_suffix="",
    enable_parity=False,
)
```

The function returns the generated module as a string and writes the file when `output_file` is provided. The `master_versions` and `slave_versions` lists are the only way to request APB5 ports or the mixed 2×2 variant; the CLI is kept narrow because mixed-version generation is normally driven by `generate_xbars.py`.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

11 Parameter Semantics

The generator resolves two classes of parameters: **generation-time** (fixed in the emitted module) and **elaboration-time** (left as `parameter` in the generated SystemVerilog).

11.1 Generation-time parameters

These are baked into the module name, the instance names, and the address map comments.

Parameter	Effect in emitted RTL
`num_masters`	Number of master APB interfaces, master-side `apb4_slave`/`apb5_slave` instances, and width of per-slave request/grant vectors.
`num_slaves`	Number of slave APB interfaces, slave-side `apb4_master`/`apb5_master` instances, and number of arbitration blocks.
`base_addr`	Stamped into `BASE_ADDR` default and the address-map comment.
`slave_size`	Stride between slaves; also sets the bit slice used for slave selection. Must be a power of two.
`master_versions` / `slave_versions`	Selects `apb4_*` vs `apb5_*` boundary modules and the presence of sideband/parity pins.
`enable_parity`	Adds APB5 parity ports when the corresponding port is APB5.
`name_suffix`	Appended to the module and file names (e.g., `_mixed`).

11.2 Elaboration-time parameters

The generated module exposes three parameters:

```
module apbx_xbar_2to4 #(
    parameter int ADDR_WIDTH = 32,
    parameter int DATA_WIDTH = 32,
    parameter int STRB_WIDTH = DATA_WIDTH / 8,
    parameter logic [ADDR_WIDTH-1:0] BASE_ADDR = 32'h10000000
) (
```

Parameter	Typical use
`ADDR_WIDTH`	Widen or narrow PADDR to match a subsystem.
`DATA_WIDTH`	32 for standard APB; 64 for wide APB variants.
`STRB_WIDTH`	Always `DATA_WIDTH / 8`; kept as a derived parameter for readability.
`BASE_ADDR`	Override the base address at instantiation time without regenerating the module. The address-map comment still reflects the generator's default.

The number of masters and slaves is **not** parameterized: a 2×4 module cannot be instantiated as 2×8. Generate a new module instead.

12 Mixed APB4/APB5 and Parity

The generator supports mixed-version ports: any master or slave can be declared APB4 or APB5 independently. The feature is controlled entirely by the `master_versions` and `slave_versions` lists.

12.1 Version list rules

- Each list has length `num_masters` or `num_slaves`.
- Valid values are `'apb4'` or `'apb5'`.
- APB4 ports expose the classic PSEL/PENABLE/PADDR/PWRITE/PWDATA/PSTRB/PPROT/PRDATA/PSLVERR/PREADY set.
- APB5 ports add `PAUSER`, `PWUSER`, `PWAKEUP`, `PRUSER`, and `PBUSER`.

12.2 PWAKEUP direction and forwarding

PWAKEUP is a **Requester** output (AMBA APB IHI 0024E section 3.7.2, Table 3-7). The crossbar sits between two kinds of APB5 interface, so the pin points a different way on each:

Port	Direction at the crossbar	Behaviour
m<i>_apb_PWAKEUP (APB5 requester port)	Input	Driven by the external requester. Parity-checked by the apb5_slave boundary IP when parity is on.
s<j>_apb_PWAKEUP (APB5 completer port)	Output	Driven by the crossbar, which is the Requester on this interface.

Each APB5 completer port drives

```
s<j>_apb_PWAKEUP = <boundary apb5_master's own PWAKEUP>
                 | m<i>_apb_PWAKEUP   for every APB5 requester i
```

- **The boundary term** is the apb5_master IP's registered wake. It is high while a command for that completer is pending or a transfer is in flight, so it leads PSEL by a cycle and holds through PREADY. It covers every transfer, including one from an APB4 requester that has no PWAKEUP at all. 3.7.2 permits a Completer to wait for PWAKEUP before asserting PREADY, so a PSEL without it could deadlock.
- **The requester terms** forward each APB5 requester's wake. Every requester can address every completer here, because there is one address map for all masters. "Any requester that can address this completer" is therefore all of them. The wake is not routed by decode: the address is unknown until PSEL, and 3.7.2 recommends PWAKEUP lead PSEL. The cost is that a requester's wake also raises PWAKEUP on completers it will not access. 3.7.2 permits a wake with no transfer, though it does not recommend one.
- **Glitch-free.** 3.7.2 requires PWAKEUP to be glitch-free and names a glitch-free OR tree as a valid construction. The tree ORs a flop output with requester PWAKEUP inputs, and those inputs are themselves required to be glitch-free.

> Nothing is registered, so a requester's wake reaches the completer in the same cycle and keeps its lead over PSEL.

The completer's PWAKEUP is not forwarded back to any requester, because it has no completer-to-requester meaning. The old crossbar treated PWAKEUP as completer-driven and dropped it (the `rsp_pwakeup` and `wakeup_request` boundary pins). Both pins are gone from `apb5_master` and `apb5_slave` (issue #102 item 7).

12.3 Mixed-port sideband behavior

The cmd/rsp fabric between boundary modules is where the protocol conversion happens. Sideband signals are carried only when **both** the source master and the target slave are APB5:

- If the source master is APB5 but the selected slave is APB4, `cmd_pauser` and `cmd_pwuser` are dropped at the slave-side APB4 master.
- If the source master is APB4 but the selected slave is APB5, the APB5 master ties `cmd_pauser` and `cmd_pwuser` to zero.
- Response sideband (`pruser`, `pbuser`) follows the same rule in reverse.

This keeps the fabric simple: there is one cmd/rsp width per direction, and the boundary modules handle expansion or zero-padding.

12.4 Parity (enable_parity=True)

APB5 parity is optional and independent of the version lists. The generator emits parity pins only on APB5 interfaces. The pins are the IHI 0024E chapter 5 check signals. Parity is **odd**: each check bit plus the bits it covers hold an odd number of ones. Wide signals get one check bit per 8 bits (Table 5-1). For a one-bit signal, the check bit is the inversion of the signal.

Check signal	Width at the crossbar	Covers	Requester port (m<i>_)	Completer port (s<j>_)
PADDRCHK	(ADDR_WIDTH+7)/8	PADDR	input, checked	output, regenerated
PCTRLCHK	1	PPROT, PWRITE (no PNSE is carried)	input, checked	output, regenerated
PSELCHK	1	PSEL (PSELxCHK)	input, checked	output, regenerated
PENABLECHK	1	PENABLE	input, checked	output, regenerated
PWDATACHK	STRB_WIDTH	PWDATA	input, checked	output, regenerated
PSTRBCHK	1	PSTRB	input, checked	output, regenerated
PAUSERCHK	1	PAUSER	input, checked	output, regenerated
PWUSERCHK	1	PWUSER	input, checked	output, regenerated
PWAKEUPCHK	1	PWAKEUP	input, checked	output, ~s<j>_apb_PWAKEUP
PRDATACHK	STRB_WIDTH	PRDATA	output, regenerated	input, checked
PREADYCHK	1	PREADY	output, regenerated	input, checked

Check signal	Width at the crossbar	Covers	Requester port (m<i>_)	Completer port (s<j>_)
PSLVERRCHK	1	PSLVERR	output, regenerated	input, checked
PRUSERCHK	1	PRUSER	output, regenerated	input, checked
PBUSERCHK	1	PBUSER	output, regenerated	input, checked

The user signals are 1 bit on the crossbar surface, so each user check signal is 1 bit. A completer port's PWAKEUPCHK is computed by the crossbar over the OR above. It is not taken from the apb5_master IP, whose own check bit covers only the IP's term, so that output is left unconnected.

Parity does **not** cross the cmd/rsp fabric. The boundary IP deconstructs the APB transfer, checks parity at entry under each signal's Table 5-1 Check Enable term, and regenerates it at exit. Each APB5 requester port exposes m<i>_parity_error_wdata and m<i>_parity_error_ctrl. Each APB5 completer port exposes s<j>_parity_error_rdata and s<j>_parity_error_ctrl. A fabric fault is therefore distinguishable from a slave's own PSLVERR response. When parity is off, the boundary IP's check inputs are tied LOW and its check outputs are left unconnected. The crossbar module then has no check pins at all.

test_apbx_xbar_parity.py generates an all-APB5 2x2 variant with enable_parity=True into its build directory. It checks every check output for odd parity, injects bad PWDATACHK and PRDATACHK and expects the matching error flag, and checks PWAKEUP forwarding from either requester to both completers.

The shipped apbx_xbar_2to2_mixed variant uses APB5 without parity. Parity is reserved for custom subsystems that explicitly request it.

13 Validation Rules

The generator validates inputs early and raises `ValueError` before emitting any RTL.

13.1 Structural checks

Rule	Error message shape
`1 <= num_masters <= 16`	“Number of masters must be 1-16, got N”
`1 <= num_slaves <= 16`	“Number of slaves must be 1-16, got N”
`len(master_versions) == num_masters`	“version list length mismatch”
`len(slave_versions) == num_slaves`	“version list length mismatch”
Each version is `'apb4'` or `'apb5'`	“unknown port version ‘…’”

13.2 Address-map checks

Rule	Rationale
`slave_size` must be a power of two.	Slave selection uses bit slicing after subtracting `BASE_ADDR`.
`slave_size >= 256` (0x100).	Keeps the stride above the smallest practical APB register region.

The generator does **not** check whether the full span (`num_slaves * slave_size`) overflows the address width; that is the caller's responsibility.

13.3 Parity checks

There is no explicit parity validation beyond the version lists. If `enable_parity=True` and a port is APB4, the generator simply omits parity pins for that port. This is intentional: mixed parity-on/off systems are allowed, and the absence of pins is the safe default.

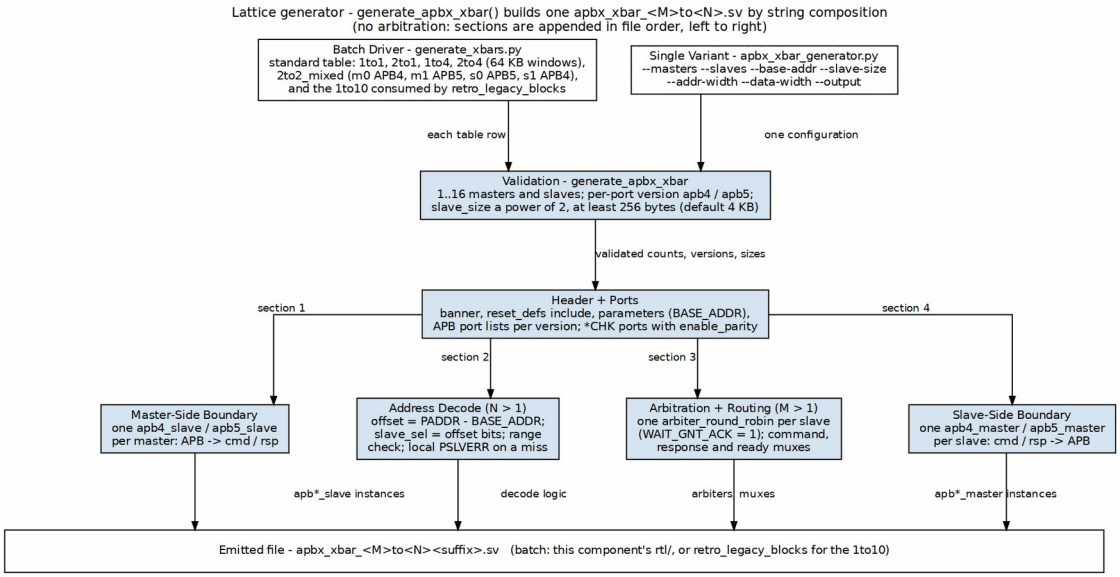
RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

14 Pipeline Overview

The generator runs in a single Python function (`generate_apbx_xbar`) and emits one SystemVerilog file. There is no separate AST framework; the output is built by string composition.

14.0.1 Figure 3.1: Lattice Generator Pipeline



Lattice Generator Pipeline

Either the batch driver (one call per row of its variant table) or the single-variant command line feeds `generate_apbx_xbar()`, which validates the counts, versions and window size, writes the header and ports, then appends four sections in file order: the master-side boundary IP, the address decode (only when there is more than one slave), the per-slave arbitration and routing (only when there is more than one master), and the slave-side boundary IP. Nothing in the generator arbitrates, so the figure has no trapezium.

14.1 Stages

1. **Validation** — counts, version lists, and `slave_size` checks.
2. **Header emission** — SPDX banner, module name, parameter list, port list, and address-map comment.
3. **Master-side boundary instantiation** — one `apb4_slave` or `apb5_slave` per master with its cmd/rsp nets.
4. **Request decode** — per-master address-to-slave index calculation (offset from `BASE_ADDR`).
5. **Arbitration and routing** — per-slave round-robin arbiter, command mux, response mux, and ready routing.
6. **Slave-side boundary instantiation** — one `apb4_master` or `apb5_master` per slave.
7. **Footer** — `endmodule` and module-name label.

The batch script `generate_xbars.py` loops over the standard variant table and calls the generator once per variant.

---

15 Request Decode

Each master presents a cmd/rsp command after the master-side `apb4_slave`/`apb5_slave` has accepted the APB transaction. The generator decodes the command's `cmd_paddr` to determine which slave (if any) should service it.

15.1 Offset-based slave selection

The slave index is computed from the offset (PADDR - BASE_ADDR), not from raw PADDR bits:

```
m0_cmd_offset    = m0_cmd_paddr - BASE_ADDR;
m0_addr_in_range = (m0_cmd_paddr >= BASE_ADDR) &&
                   (m0_cmd_paddr < (BASE_ADDR + 32'h00040000));
m0_slave_sel     = m0_cmd_offset[17:16];
```

Using the offset prevents the address map from rotating when BASE_ADDR is not aligned to the total span. The subtraction folds to constants at elaboration because BASE_ADDR is a parameter, so it costs no logic.

15.2 Decode-miss handling

If a command falls outside the mapped range, the crossbar must complete the transaction with PSLVERR rather than holding PREADY low forever. The generator emits a per-master `r_m<N>_decerr_pending` flop:

- When `cmd_valid && cmd_ready && !addr_in_range`, the pending flag is set.
- The master-side `cmd_ready` is asserted only while the pending flag is clear.
- A locally generated response (`rsp_valid = 1`, `rsp_pslverr = 1`) is returned on the next cycle and cleared when `rsp_ready` is asserted.

This guarantees that an unmapped access completes as a recoverable error instead of wedging the external master.

15.3 Single-master simplification

When `num_masters == 1`, no arbitration is needed. The request decode still creates the per-slave `cmd_valid` signals directly from the slave-select result, and the decode-miss logic remains.

---

16 Arbitration and Routing

For each slave, the generator emits an independent round-robin arbiter that selects among the masters requesting that slave.

16.1 Per-slave request vector

```
s0_arb_request[0] = m0_cmd_valid && m0_addr_in_range && (m0_slave_sel
== 2'd0);
s0_arb_request[1] = m1_cmd_valid && m1_addr_in_range && (m1_slave_sel
== 2'd0);
```

When `num_slaves == 1`, the slave-select check is omitted.

16.2 Round-robin arbitration

The generator instantiates `arbiter_round_robin` with `WAIT_GNT_ACK=1` so the grant is locked until the transaction completes:

```
arbiter_round_robin #(
    .CLIENTS(2),
    .WAIT_GNT_ACK(1)
) u_s0_arbiter (...);
```

The grant-ack vector is driven by `rsp_valid && rsp_ready` on the selected slave port, ensuring the grant persists across the entire APB access.

16.3 Command routing

A combinational `case (1'b1)` mux routes the granted master's cmd fields to the slave-side cmd nets. APB5 sideband is carried only when both the master and the slave are APB5.

16.4 Response routing

The generator registers each master's slave selection (`r_m<N>_slave_sel`) when a command is accepted. On the response path, the registered slave index selects which slave's `rsp_*` signals are returned to the master. The registered selection keeps the response aligned with the command even if a new command arrives before the first response completes.

16.5 Ready routing

- Master `cmd_ready` is asserted only by the slave selected by the current command.
- Slave `rsp_ready` is asserted only by the master currently holding the grant for that slave.
- The decode-miss path bypasses the slave arbiters and completes locally.

17 Boundary Module Instantiation

The generated module instantiates proven boundary modules from the `rtl/common/` and `rtl/amba/` libraries rather than reimplementing APB protocol logic inline.

17.1 Master-side modules

For each master `m`:

- `apb4_slave` when `master_versions[m] == 'apb4'`.
- `apb5_slave` when `master_versions[m] == 'apb5'`.

Parameterization:

```
apb4_slave #(
    .ADDR_WIDTH(ADDR_WIDTH),
    .DATA_WIDTH(DATA_WIDTH),
    .STRB_WIDTH(STRB_WIDTH),
    .PROT_WIDTH(3)
) u_apb4_slave_m0 (...);
```

APB5 variants add user widths of 1 and, when parity is enabled, `ENABLE_PARITY=1`.

17.2 Slave-side modules

For each slave `s`:

- `apb4_master` when `slave_versions[s] == 'apb4'`.
- `apb5_master` when `slave_versions[s] == 'apb5'`.

The same width and sideband parameter rules apply.

17.3 Parity tie-offs

When parity is disabled on an APB5 port, the `*CHK` input pins are tied to zero and the `*CHK` output and `parity_error_*` pins are left unconnected. This lets the same `apb5_slave`/`apb5_master` modules be reused with or without parity without conditional module selection.

17.4 PWAKEUP wiring

An APB5 master port's PWAKEUP input goes to the `apb5_slave` boundary IP, which uses it only for the PWAKEUPCHK check. Each `apb5_master` boundary IP drives its own registered wake onto an internal net, `w_s<s>_ip_pwakeup`. The generator then emits `assign s<s>_apb_PWAKEUP = w_s<s>_ip_pwakeup | m<i>_apb_PWAKEUP ...`, with one term for every APB5 master. With parity on, it also emits `assign s<s>_apb_PWAKEUPCHK = ~s<s>_apb_PWAKEUP`. The IP's own `m_apb_PWAKEUPCHK` is left unconnected, because it covers only the IP's term.

17.5 1×1 passthrough

When `num_masters == 1` and `num_slaves == 1`, the fabric reduces to simple `assign` statements between the single master-side and single slave-side cmd/rsp nets. No arbiter is instantiated.

18 Directory Layout

The generator writes a single `.sv` file per invocation. The standard family is organized as follows:

```
lattice-apbx-xbar/
├── bin/
│   ├── apbx_xbar_generator.py      # low-level generator
│   └── generate_xbars.py           # batch generator
├── rtl/
│   ├── apbx_xbar_1to1.sv
│   ├── apbx_xbar_2to1.sv
│   ├── apbx_xbar_1to4.sv
│   ├── apbx_xbar_2to4.sv
│   └── apbx_xbar_2to2_mixed.sv
└── dv/
    └── tests/                      # cocotb test suite
```

The Retro Legacy Blocks 1-to-10 variant is emitted outside this tree because it is consumed by another component:

```
retro_legacy_blocks/rtl/apbx_xbar/apbx_xbar_1to10.sv
```

This external placement is intentional; it prevents the RLB crossbar from drifting away from the family fixes.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

19 Module Header and Parameters

Every generated file begins with the standard RTL Design Sherpa banner and includes `reset_defs.svh`.

```
// SPDX-License-Identifier: MIT
// SPDX-FileCopyrightText: 2024-2025 sean galloway
//
// RTL Design Sherpa ...
//
// Module: apbx_xbar_2to4
// Purpose: Apbx Xbar 2to4 module
//
// Documentation: docs/markdown/rtl-amba/index.md
// Subsystem: amba
//
// Author: sean galloway
// Created: 2025-10-18

`timescale 1ns / 1ps
`include "reset_defs.svh"
```

19.1 Address-map comment

Immediately before the module declaration, the generator emits a comment block documenting the slave map:

```
//   Slave 0: [0x10000000, 0x1000FFFF] (64KB)
//   Slave 1: [0x10010000, 0x1001FFFF] (64KB)
//   ...
```

This comment is derived from `base_addr` and `slave_size` and is the authoritative map for integrators.

19.2 Port naming convention

Direction	Naming
Master-side inputs	`m<N>_apb_<signal>`
Master-side outputs	`m<N>_apb_<signal>`
Slave-side outputs	`s<N>_apb_<signal>`
Slave-side inputs	`s<N>_apb_<signal>`

Parity error outputs (APB5 with parity) use `m<N>_parity_error_wdata` / `m<N>_parity_error_ctrl` on the master side and `s<N>_parity_error_rdata` / `s<N>_parity_error_ctrl` on the slave side.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

20 cmd/rsp Fabric

Between the APB boundary modules, the generated module implements a command/response fabric with the following signals per master and per slave:

Signal	Direction from master-side	Width	Meaning
`cmd_valid`	master → fabric	1	Command available.
`cmd_ready`	fabric → master	1	Command accepted.
`cmd_pwrite`	master → fabric	1	Write (1) or read (0).
`cmd_paddr`	master → fabric	`ADDR_WIDTH`	Address.
`cmd_pwdata`	master → fabric	`DATA_WIDTH`	Write data.
`cmd_pstrb`	master → fabric	`STRB_WIDTH`	Byte strobe.
`cmd_pprot`	master → fabric	3	Protection.
`cmd_pauser`	master → fabric	1	APB5 sideband (only on APB5 master).
`cmd_pwuser`	master → fabric	1	APB5 sideband (only on APB5 master).
`rsp_valid`	fabric → master	1	Response available.
`rsp_ready`	master → fabric	1	Response accepted.
`rsp_prdata`	fabric → master	`DATA_WIDTH`	Read data.
`rsp_pslverr`	fabric → master	1	Slave error.
`rsp_pruse`	fabric → master	1	APB5

Signal	Direction from master-side	Width	Meaning
`r`			sideband (only on APB5 master).
`rsp_pbuser`	fabric → master	1	APB5 sideband (only on APB5 master).

The slave-side cmd/rsp signals have the same names and directions reversed relative to the slave-side APB master modules.

20.1 Fabric width invariance

The cmd/rsp width is fixed regardless of mixed APB4/APB5 ports. APB4 ports simply do not drive or receive the sideband signals; APB4 slaves tie them to zero and APB4 masters ignore them. This avoids parameterized struct widths inside the fabric.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

21 Wrapper Scaffolds

The generator emits only the core crossbar module. Wrapper scaffolds for testbench convenience exist only for the four legacy fixtures and are **not** regenerated by `generate_xbars.py`. If a new variant needs a wrapper, create it by hand in the DV area or add a wrapper emission stage to the generator.

21.1 Filelist convention

Area flows that consume the generated modules typically reference the RTL directly rather than a generated `.f` file. An example filelist for the standard variants would contain:

```
rtl/amba/apb4/xbar/apbx_xbar_2to4.sv
rtl/common/arbiter_round_robin.sv
rtl/common/apb4_slave.sv
rtl/common/apb4_master.sv
```

The exact dependencies are determined by the variant (APB5 ports add `apb5_slave.sv` and `apb5_master.sv`). The component-level `Makefile` or area regression flow maintains the canonical filelist.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

22 Full Regeneration Rule

Any change to the files listed below must be followed by a full regeneration of every standard variant and a full regression run.

Source	Why it forces regeneration
`bin/apbx_xbar_generator.py`	Every emitted line comes from this file.
`bin/generate_xbars.py`	Drives the variant table and default parameters.
`rtl/common/apb4_slave.sv` / `apb5_slave.sv`	Boundary module behavior changes propagate into every master-side instance.
`rtl/common/apb4_master.sv` / `apb5_master.sv`	Same for slave-side instances.
`rtl/common/arbiter_round_robin.sv`	Arbitration timing or fairness changes affect every multi-master variant.

22.1 Regeneration command

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin
python generate_xbars.py
```

After regeneration, the only expected `git diff` should be in `rtl/*.sv` and, if the RLB variant was affected, `retro_legacy_blocks/rtl/apbx_xbar/apbx_xbar_1to10.sv`.

22.2 Do not hand-edit generated files

The `.sv` files in `rtl/` are generator output. Hand-edits will be overwritten on the next regeneration and are not reviewable as source changes. Fix the generator instead.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

23 Pre-Generated Fixtures

The component ships with five pre-generated fixtures:

File	Masters	Slaves	Notes
`apbx_xbar_1to1.sv`	1	1	Simple passthrough; no arbiter.
`apbx_xbar_2to1.sv`	2	1	Arbitration only, no address decode.
`apbx_xbar_1to4.sv`	1	4	Address decode only, no arbitration.
`apbx_xbar_2to4.sv`	2	4	Full crossbar: decode + arbitration.
`apbx_xbar_2to2_mixed.sv`	2	2	APB4/APB5 mixed-port boundary test.

These fixtures are the regression baseline. They are generated with:

- `base_addr = 0x10000000`
- `slave_size = 0x10000` (64KB windows)
- `addr_width = 32`, `data_width = 32`

The mixed variant uses `master_versions=['apb4','apb5']` and `slave_versions=['apb5','apb4']`.

23.1 When to add a new fixture

Add a new standard fixture only when:

1. A new size is required by multiple subsystems, or
2. A new port-version combination needs continuous regression coverage.

Add the new entry to the `variants` or `mixed` table in `generate_xbars.py`, regenerate, and add a matching cocotb test.

---

24 DV Suite

The component-level DV suite lives in `dv/tests/` and uses cocotb.

24.1 Test list

Test	Focus
`test_apbx_xbar_1to1.py`	Passthrough behavior.
`test_apbx_xbar_2to1.py`	Two masters contend for one slave.
`test_apbx_xbar_1to4.py`	Single master accesses multiple slaves by address.
`test_apbx_xbar_2to4.py`	Full decode + arbitration stress.
`test_apbx_xbar_2to2_mixed.py`	APB4/APB5 sideband conversion.
`test_apbx_xbar_timing.py`	PREADY backpressure and timing.

24.2 Running the suite

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/dv/tests
make clean-all && make run-all-gate
```

The `run-all-gate` target exercises every standard fixture through the gate-level-style regression flow. After a generator change, this must pass with zero failures before the regenerated RTL is committed.

24.3 Adding a new test

1. Add a new `test_apbx_xbar_<variant>.py` file in `dv/tests/`.
2. Reference the matching generated module in the test's `Makefile` or parameter list.
3. Add the test name to the area regression list (component `Makefile` or `tests.mk`).

25 Lint Checks

The generated RTL must pass the project's lint flow. The most common lint concerns for generated crossbars are width mismatches and unused signals.

25.1 Width checks

- `slave_addr_bits` is `ceil(log2(num_slaves))` and is at least 1.
- `slave_offset_bits` is `log2(slave_size)` and is used to slice the offset.
- The `BASE_ADDR` parameter is `ADDR_WIDTH` bits.
- APB5 sideband signals are 1-bit logic. Check vectors are `(ADDR_WIDTH+7)/8` bits for `PADDRCHK`, `STRB_WIDTH` bits for `PWDATACHK`/`PRDATACHK`, and 1 bit for every other `*CHK` signal.

25.2 Unused signal checks

When parity is disabled on an APB5 port, the generator leaves some output pins unconnected. The lint waiver for these is part of the boundary module, not the generated crossbar. Ensure the waiver file or lint command set covers `apb5_slave` and `apb5_master` tie-offs.

25.3 Synthesis smoke test

As a minimum sanity check, run a synthesis tool on the 2×4 variant after any generator change. The expected result is clean elaboration with no latch inference and no combinational loops.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

26 Adding a New Standard Variant

To add a new pre-generated fixture size:

1. Open `bin/generate_xbars.py`.
2. Add the `(masters, slaves)` tuple to the `variants` list for plain APB4, or add a `mixed` entry for a new version combination.
3. Add a matching cocotb test in `dv/tests/`.
4. Regenerate:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin
python generate_xbars.py
```

5. Run the DV suite:

```
cd ../dv/tests
make clean-all && make run-all-gate
```

6. Commit the generator change, the new RTL file, and the new test together.

26.1 Naming convention

- Plain variants: `apbx_xbar_<M>to<N>.sv`
- Mixed-version variants: `apbx_xbar_<M>to<N>_<suffix>.sv`

The suffix must be alphanumeric with underscores and must be reflected in the `name_suffix` argument.

---

27 Adding a New Generator Option

To add a scalar option that affects all variants (for example, a new default `slave_size`):

1. Add the argument to `apbx_xbar_generator.py`'s `argparse` block and pass it through `generate_apbx_xbar`.
2. Add the same argument to `generate_xbars.py` if users should be able to override it for the standard family.
3. Document the option in this GAS, the HAS, and the PRD if it changes the hardware behavior.
4. Add validation near the top of `generate_apbx_xbar`.
5. Regenerate and run the full DV suite.

27.1 Backward compatibility

Default values must preserve existing generated output. If the new option changes the default address map or port list, introduce it as an opt-in flag first and migrate the standard variants in a separate commit.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

28 Adding APB5 Parity or Sideband

The APB5 sideband and parity machinery is already present in the generator. Extending it usually means widening a sideband signal or changing how parity is computed.

28.1 Widening sideband signals

The generator hard-codes 1-bit user signals (`AUSER_WIDTH=1`, `WUSER_WIDTH=1`, etc.). To support wider user fields:

1. Add a new generator option (e.g., `user_width`).
2. Pass the width through to the `apb5_slave`/`apb5_master` parameterization.
3. Update the fabric cmd/rsp declarations to carry the wider fields.
4. Update the port list generation to declare the wider APB5 pins.

28.2 Changing parity behavior

The check signals are the AMBA APB (IHI 0024E) chapter 5 `*CHK` set. They use odd parity, with one bit per 8 bits of the covered signal. The boundary modules check them on entry and generate them on exit, and the crossbar only routes them. The one exception is a completer port's `PWAKEUPCHK`, which the crossbar computes as `~PWAKEUP` over its own wake OR (see *Mixed APB4/APB5 and Parity*). To change parity width or algorithm:

1. Modify `apb5_slave` and `apb5_master` to support the new scheme.
2. Update the crossbar generator's parity vector widths and tie-offs accordingly.
3. Extend `dv/tests/test_apbx_xbar_parity.py`, which generates a parity-enabled variant, checks every check output for odd parity, and injects check-bit errors against the `parity_error_*` outputs.

Remember that parity is checked and regenerated at each boundary. It never crosses the cmd/rsp fabric.

29 Testing an Extension

Every generator change must be exercised by at least one generated fixture and one DV test.

29.1 Minimal test plan

1. **Generator unit test** — call `generate_apbx_xbar` with the new option and verify the emitted string contains the expected constructs.
2. **Fixture regeneration** — run `generate_xbars.py` and verify no unexpected diffs appear in existing fixtures.
3. **Simulation test** — add or extend a cocotb test that exercises the new behavior.
4. **Lint/synthesis smoke** — run the lint flow on the affected variants.

29.2 Example: adding a custom variant test

```
# dv/tests/test_apbx_xbar_3to6.py
from apbx_xbar_test_base import ApbxXbarTestBase

class TestApbxXbar3to6(ApbxXbarTestBase):
    def test_basic_access(self):
        self.dut = cocotb.top
        # exercise all 3 masters against all 6 slaves
```

The test must be added to the `run-all-gate` target or an equivalent regression list.

29.3 Commit discipline

Commit generator source changes, regenerated RTL, and DV tests in a single logical change. Do not commit hand-edited generated files.